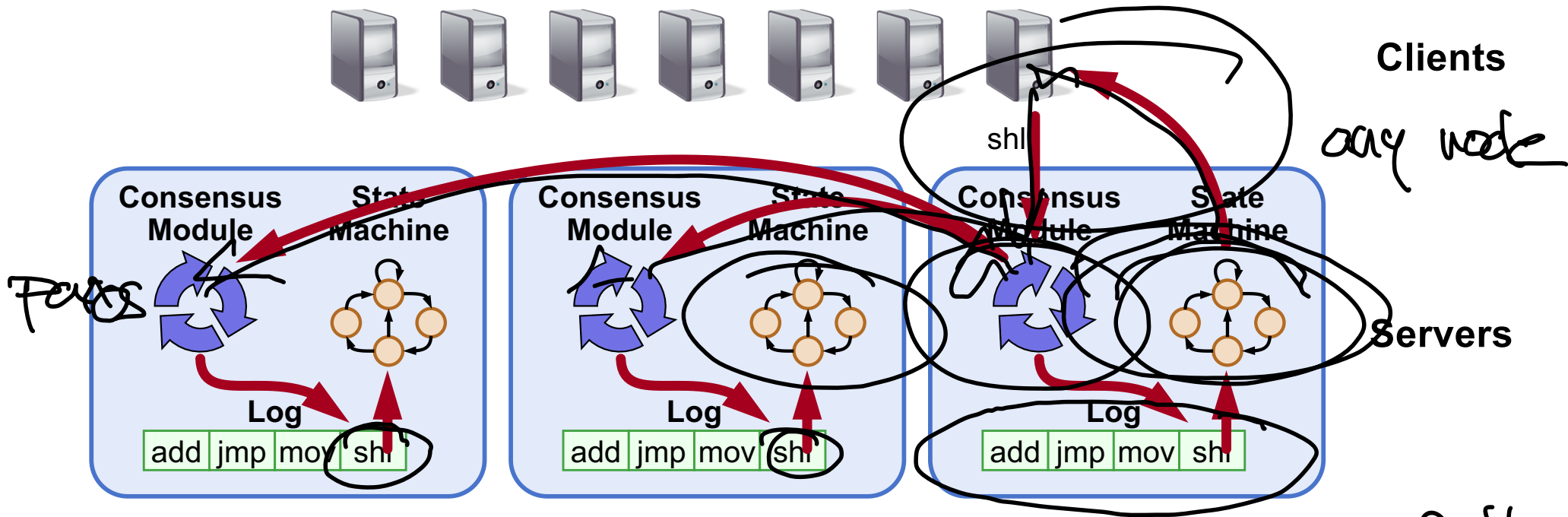

Implementing Replicated Logs with Paxos

Borrows heavily from slides by Lorenzo Alvisi, Ali Ghodsi, and David Mazières
John Ousterhout and Diego Ongaro - *Raft*

- Paxos Made Simple
- Leslie Lamport

Goal: Replicated Log



- Replicated log => **replicated state machine**
 - All servers execute same commands in same order
- Consensus module ensures proper log replication
- System makes progress as long as any majority of servers are up

The Paxos Approach

Decompose the problem:

- **Basic Paxos (“single decree”):**

- Simplest form of consensus
- Agree on a single value
 - One or more servers propose values
 - System must agree on a **single value** as **chosen** (not ‘accept’)
single node
 - Only one value is ever chosen

- **Multi-Paxos:**

- Agree on a log - position in log + value
- Combine several instances of Basic Paxos to agree on a series of values forming the log
- No clear specification
- We will read Raft

Requirements for Basic Paxos

- **Safety:** ~Correctness - Nothing bad
 - Only a single value may be chosen
 - A server never learns that a value has been chosen unless it really has been
- **Liveness (as long as majority of servers up and communicating with reasonable timeliness):**
 - Availability
 - Some proposed value is eventually chosen
 - If a value is chosen, servers eventually learn about it

Something good eventually happens

Failure model: fail-stop / fail-recover crash and restart

- Messages reordered, dropped, and duplicated, network partitions
- What can't Paxos tolerate? Byzantine

persistent storage

Paxos Components

- **Proposers:**

- Active: put forth particular values to be chosen

- **Acceptors:**

- Passive: respond to messages from proposers
- Responses represent votes that form consensus
- Store chosen value, state of the decision process
- Want to know which value was chosen – *Learners*

Clients talk to?

Any proposer

For this presentation:

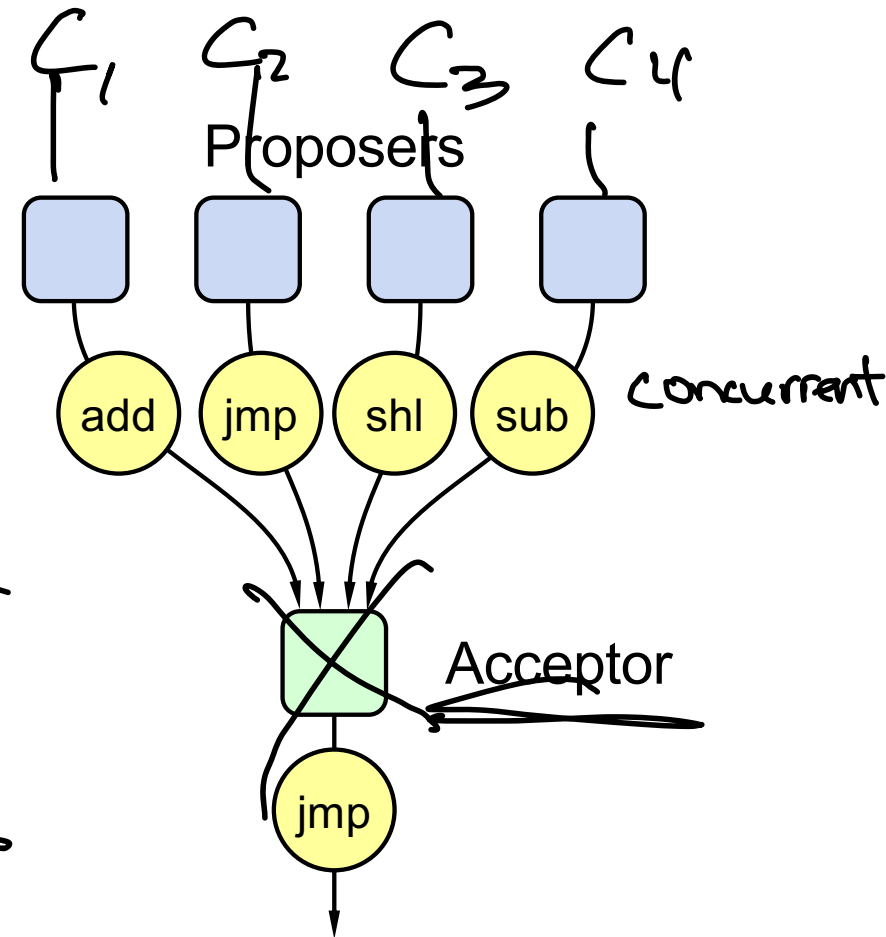
- Each Paxos server contains both components

Build toward Correct Solution

- Simple (incorrect) approach: a single acceptor chooses value
- Problems?

- single point of failure

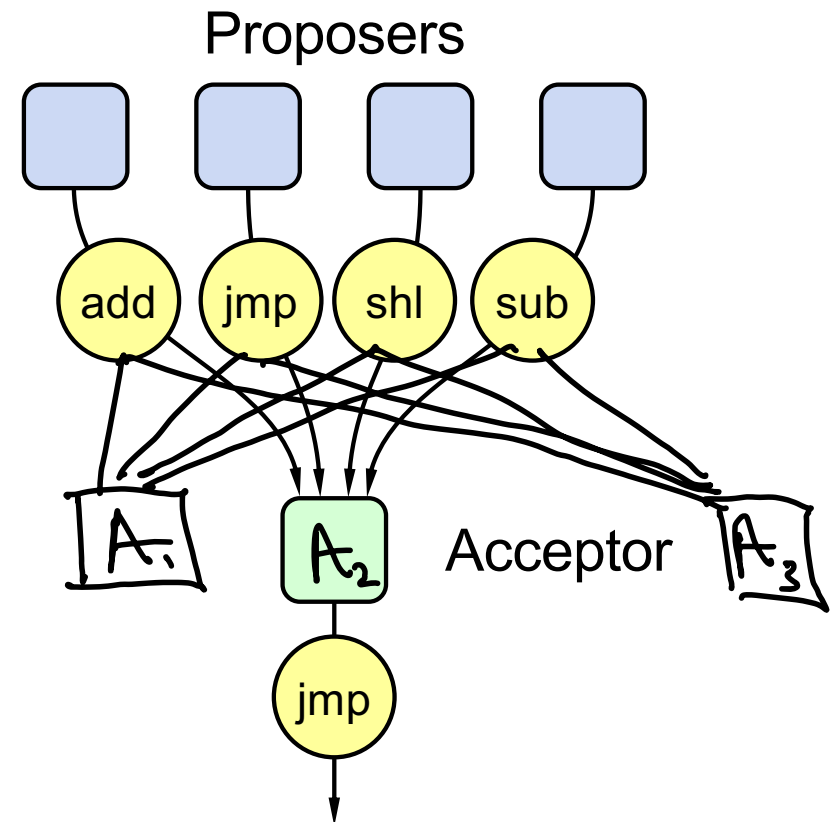
- not available if 1 acceptor crashes



Build toward Correct Solution

- Simple (incorrect) approach: a single acceptor chooses value
- What if acceptor crashes after choosing?
- Solution: quorum small odd #
 - Multiple acceptors (3, 5, ...)
 - Value v is **chosen** if **accepted** by **majority** of acceptors
 - If one acceptor crashes, chosen value still available

agreed upon

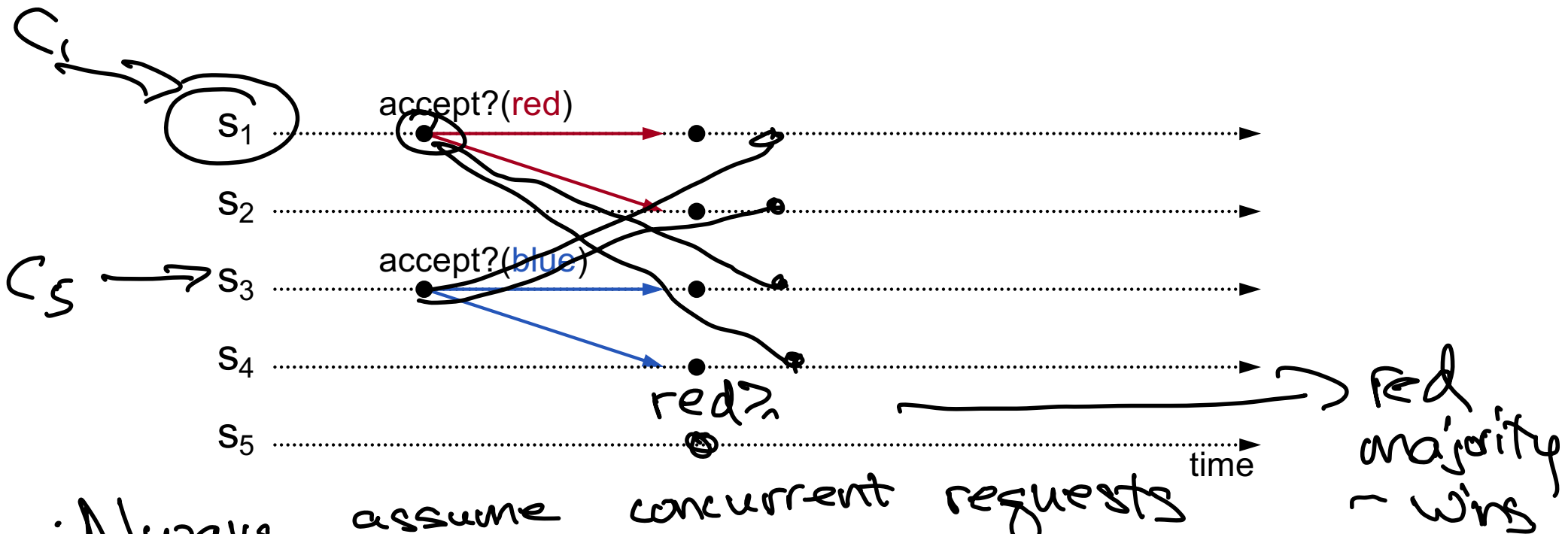


How do nodes decide which value to **accept**??

Approach 2: Accept First Proposal

- Acceptor ^{votes} accepts only first value it receives?

~~• Problem?~~ Works sometimes...



• Always assume concurrent requests

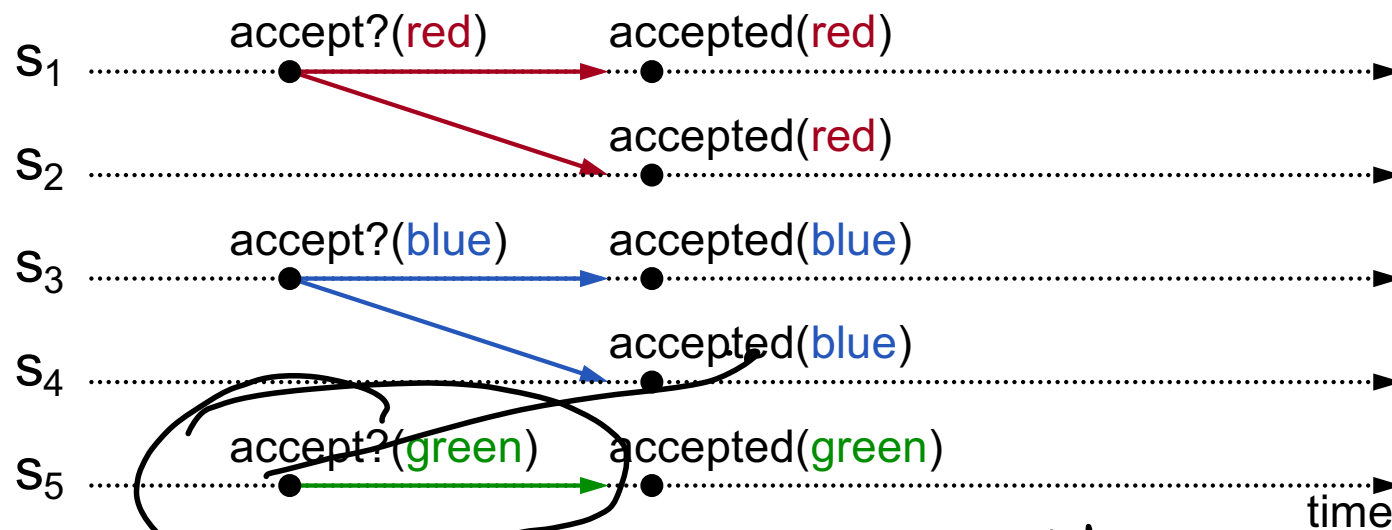
• S5 determines winner -

give majority to red/blue

• timing determines first value on each node

Approach 2: Accept First Proposal

- Acceptor accepts only first value it receives?
- If simultaneous proposals, no value might have majority



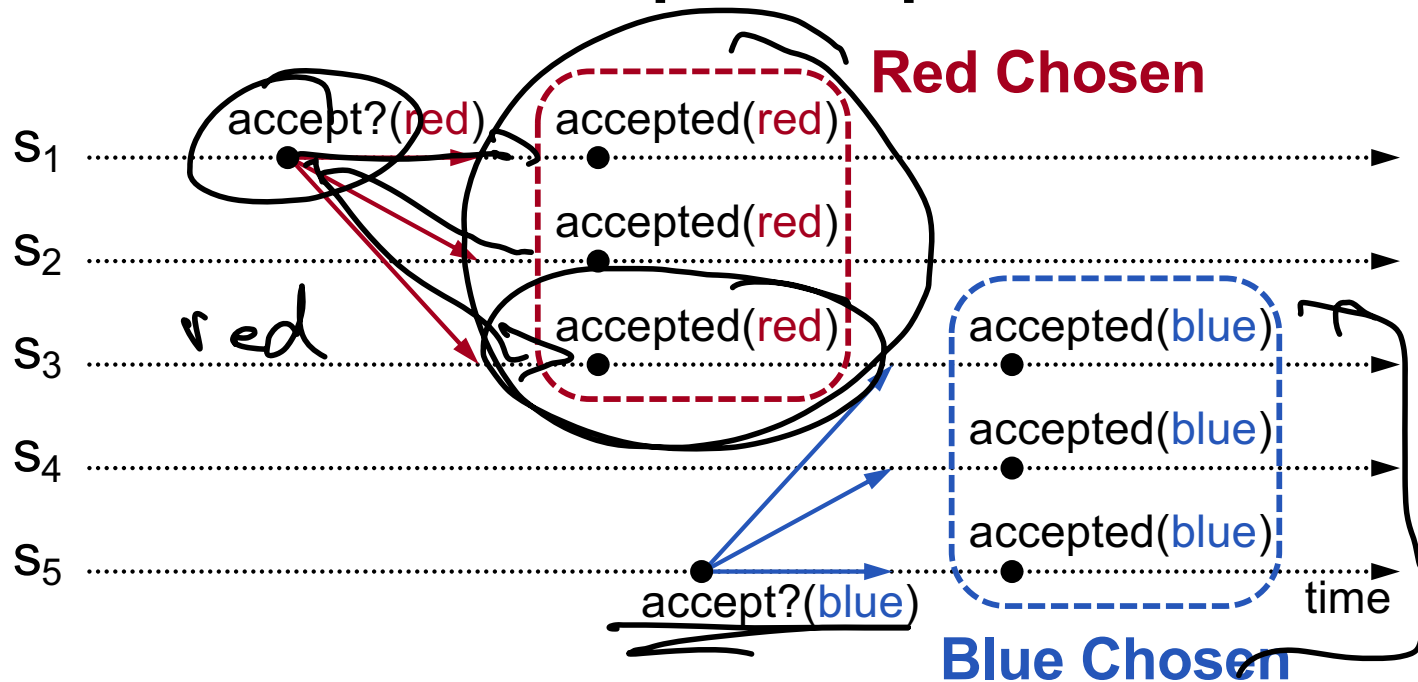
no majority of values
→ no agreement

Implication?

Acceptors must sometimes change mind

Approach 3: Accept Every Value?

- ^{votes for} Acceptor accepts **every** value it receives?
- Problem? Could accept multiple values

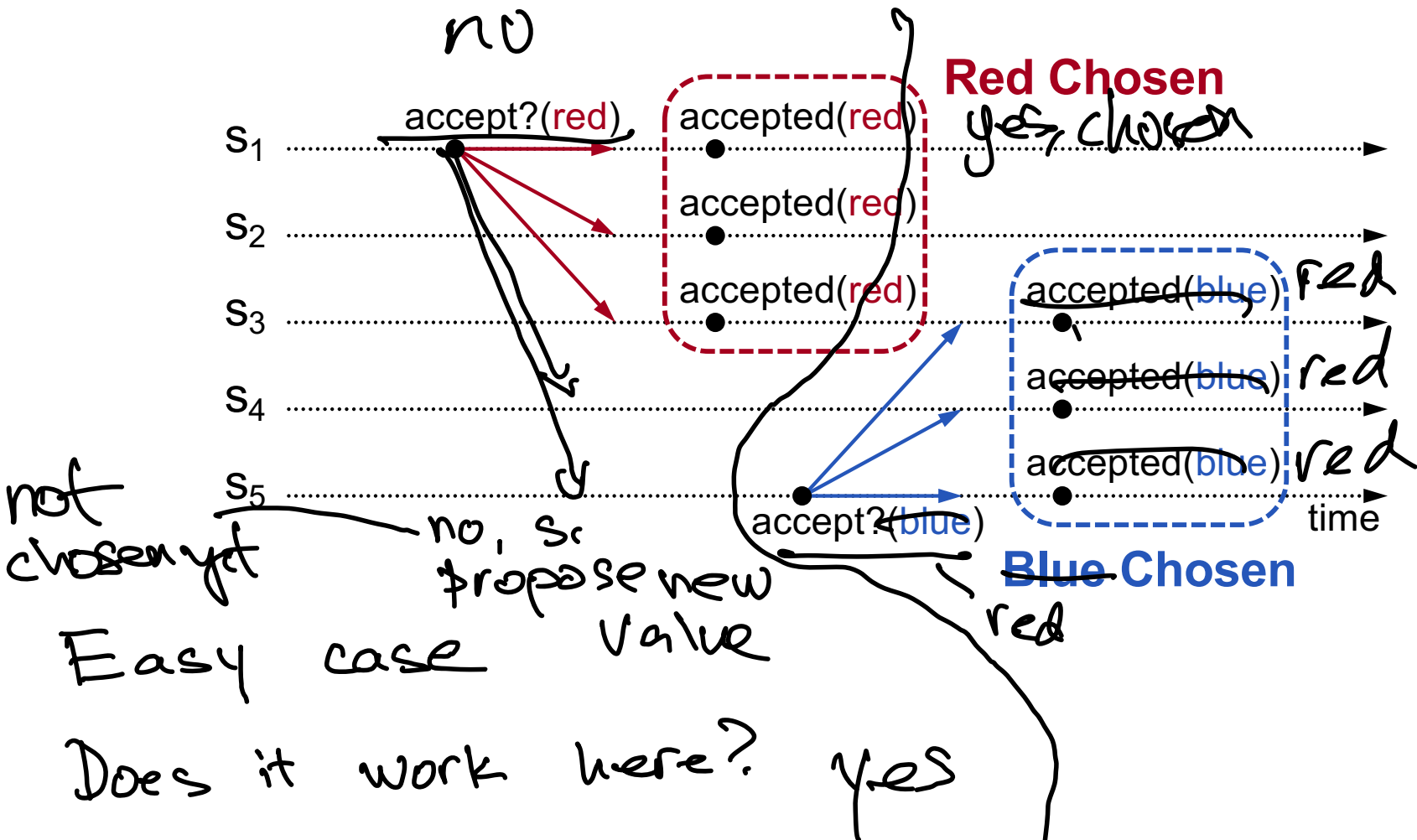


Implication? ^{has a majority}
Once a value has been chosen, future proposals must propose that same value (**2-phase protocol**) → check if chosen

Approach 4: Discover?

- Once a value has been chosen, future proposals must propose that same value (2-phase protocol)

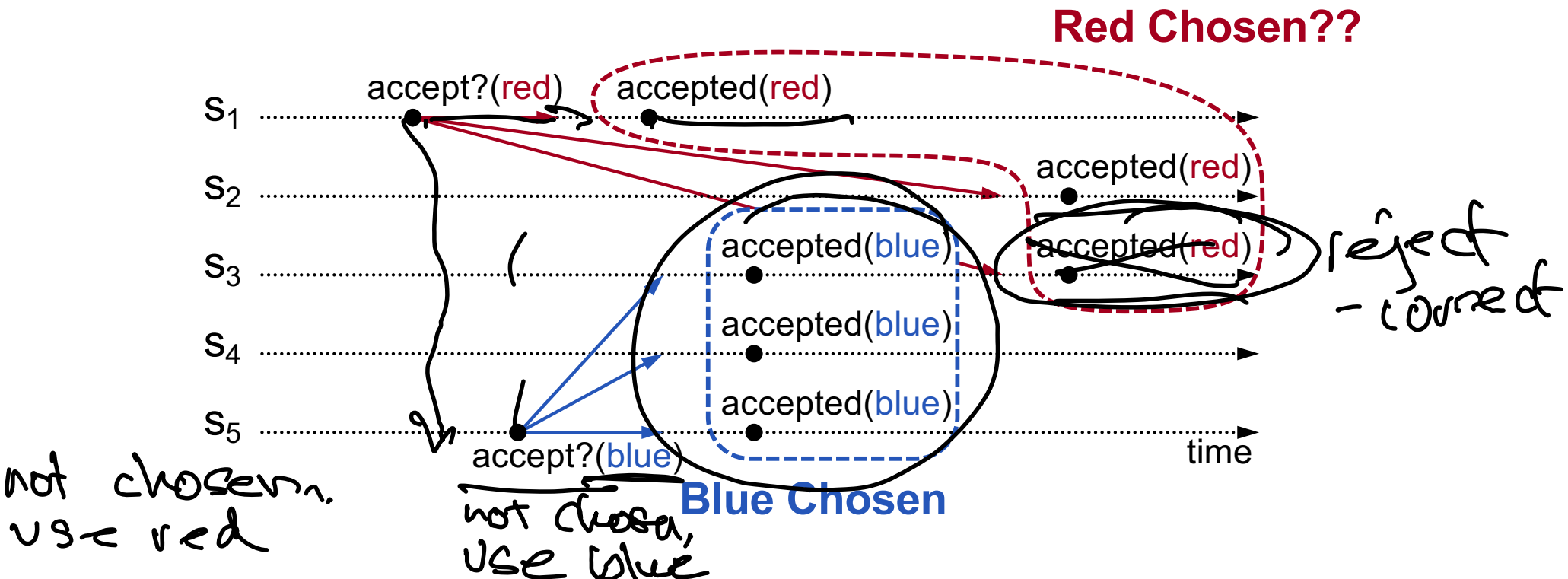
agreed on by majority



Naive alg.

Approach 4: Discover?

harder case - won't work

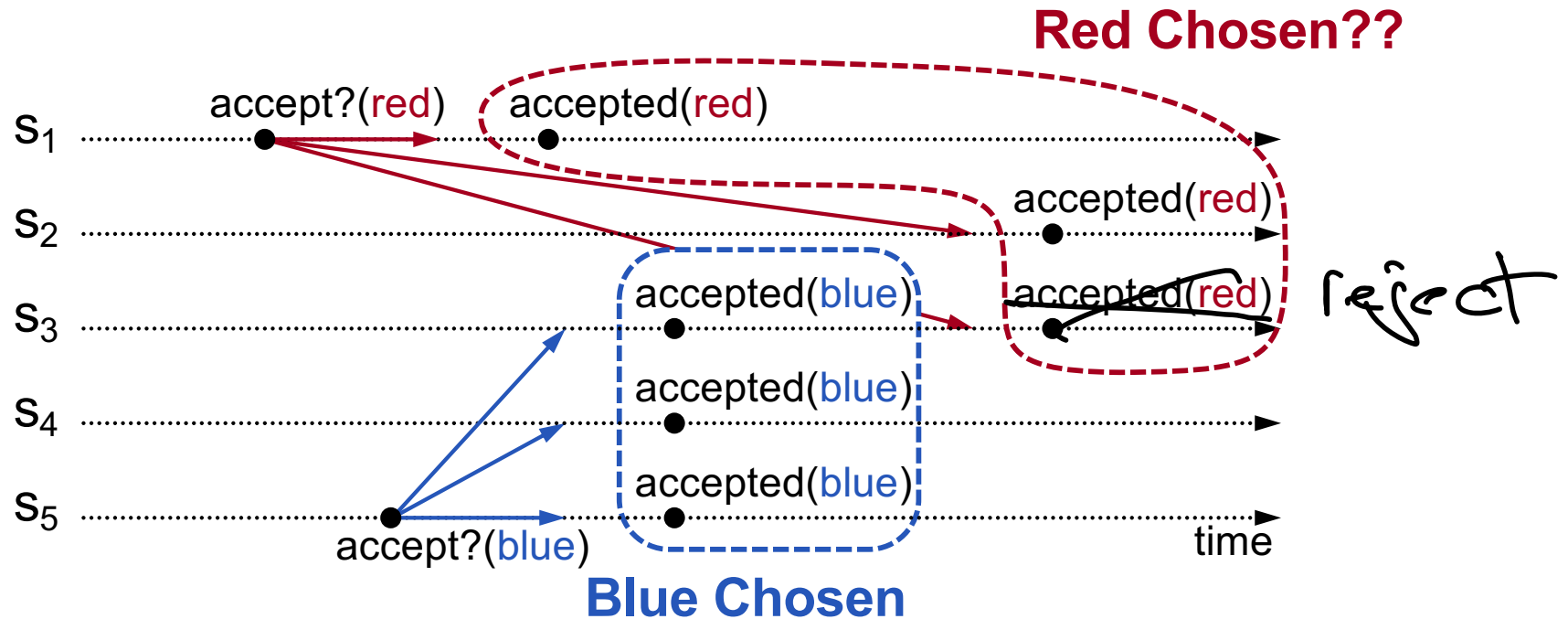


- s_5 needn't propose **red** (it hasn't been chosen yet)
- s_1 's proposal must be aborted (s_3 must reject it)

Implication?

Must **order** proposals, reject old ones

Basic Paxos



Must **order** proposals, reject old ones

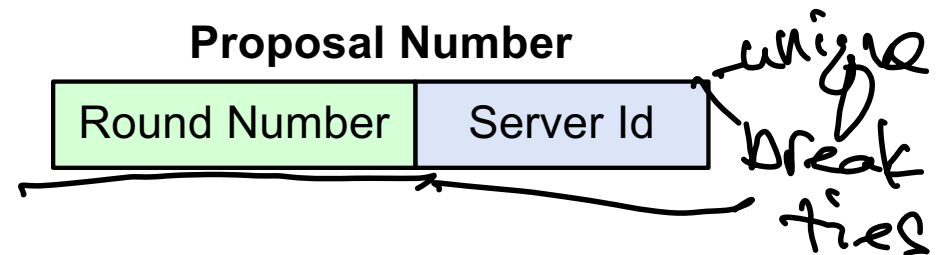
Proposal Numbers

- Each proposal has a unique number
 - Higher numbers take priority over lower numbers
 - Higher numbers → Later proposal
 - It must be possible for a proposer to choose a new proposal number higher than anything it has seen/used before

prioritize later proposals - because earlier did not complete yet

more recent

- One simple approach:



- Each server stores maxRound: the largest Round Number it has seen so far
- To generate a new proposal number:
 - Increment maxRound
 - Concatenate with Server Id
- Proposers must persist maxRound on disk: must not reuse proposal numbers after crash/restart

Basic Paxos

Two-phase approach:

- **Phase 1: broadcast **Prepare** RPCs**
 - Find out about any (potentially) chosen values
 - Block older proposals that have not yet completed
- **Phase 2: broadcast **Accept** RPCs**
 - Ask acceptors to accept a specific value

Acceptors promise: not to accept any older proposals

Value is chosen when: majority of acceptors accept your value/request

~~2~~ # nodes in system

initial value

Basic Paxos

Proposers

- 1) Choose new proposal number n
- 2) Broadcast Prepare(n) to all - up servers
at least majority

- 4) When responses received from majority:

- If any acceptedValues returned, replace value with acceptedValue for highest acceptedProposal *winner*

- 5) Broadcast Accept(n , value) to all servers

- 6) When responses received from majority: *proposal #*

- Any rejections (result > n)? goto (1) *new value*
- Otherwise, **value is chosen**

Acceptors must record minProposal, acceptedProposal, and acceptedValue on stable storage (disk)

Acceptors

- a) promise to not accept any older proposals

- 3) Respond to Prepare(n):

- If $n > \text{minProposal}$ then minProposal = n
- Return(acceptedProposal, acceptedValue)

- b) find out about any potentially chosen values

- 6) Respond to Accept(n , value):

- If $n \geq \text{minProposal}$ then
acceptedProposal = minProposal = n
acceptedValue = value
- Return(minProposal)

majority chose ours

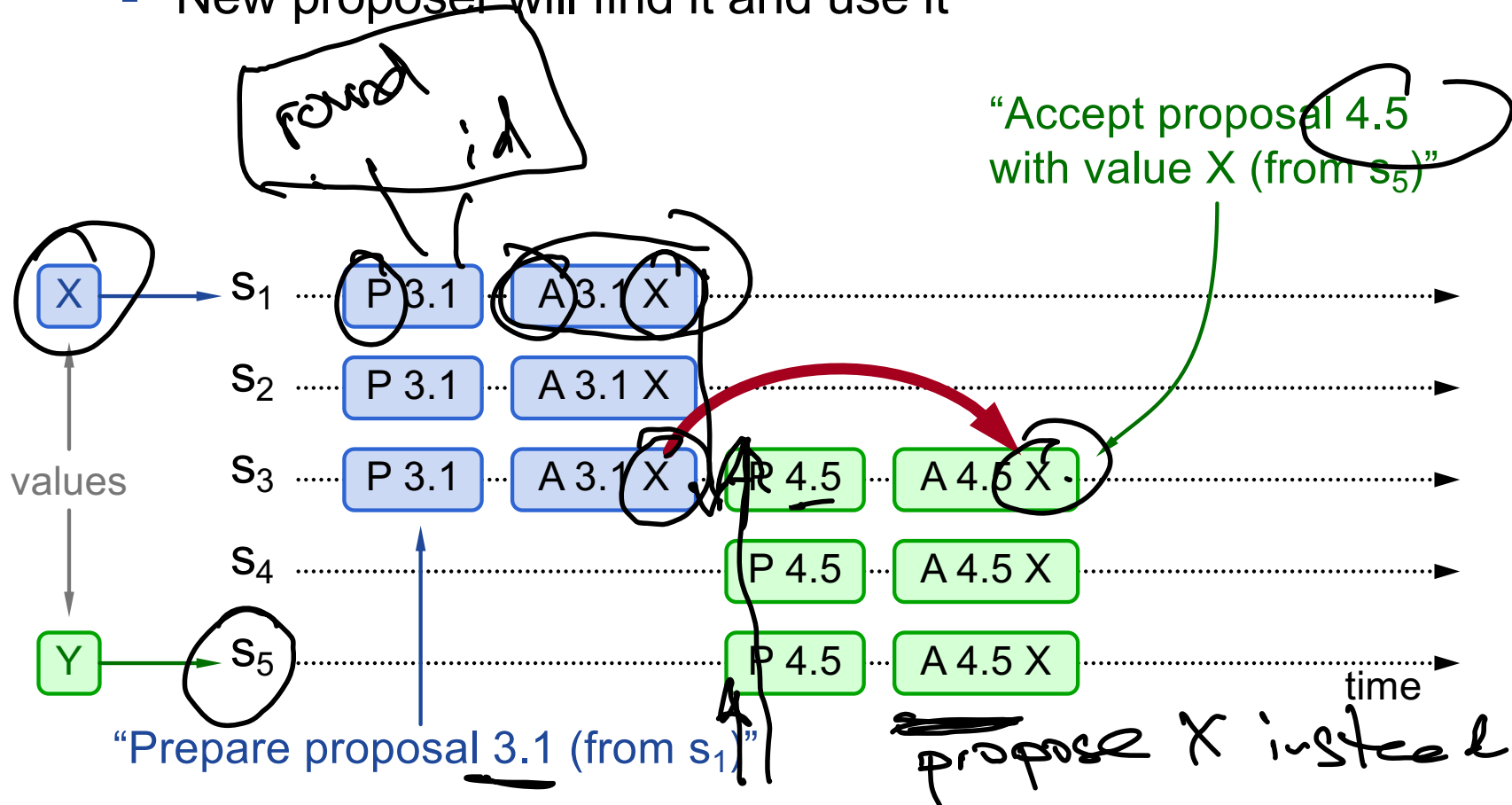
can notify all other nodes

Basic Paxos Example 1

Three possibilities when later proposal prepares:

1. Previous value already chosen:

- New proposer will find it and use it

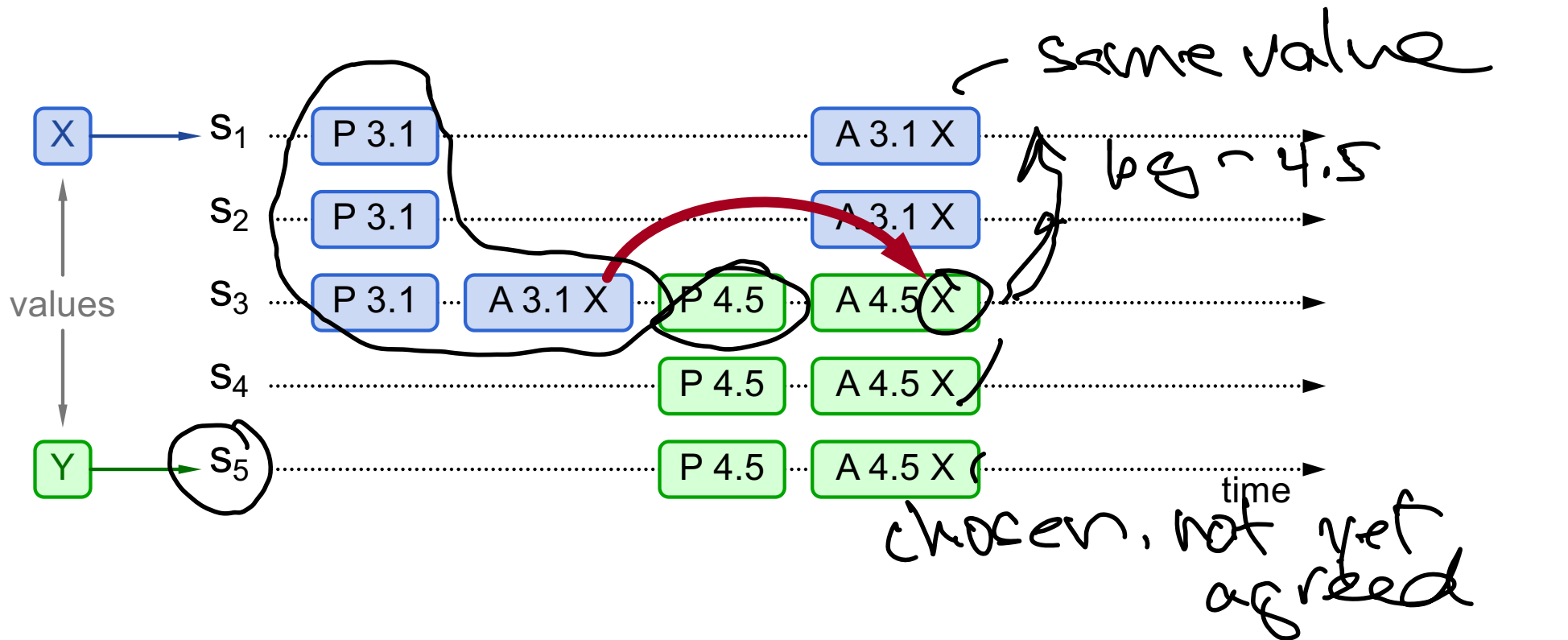


Basic Paxos Example 2

Three possibilities when later proposal prepares:

2. Previous value not chosen, but later proposer sees it:

- New proposer will use existing value
- Both proposers can succeed

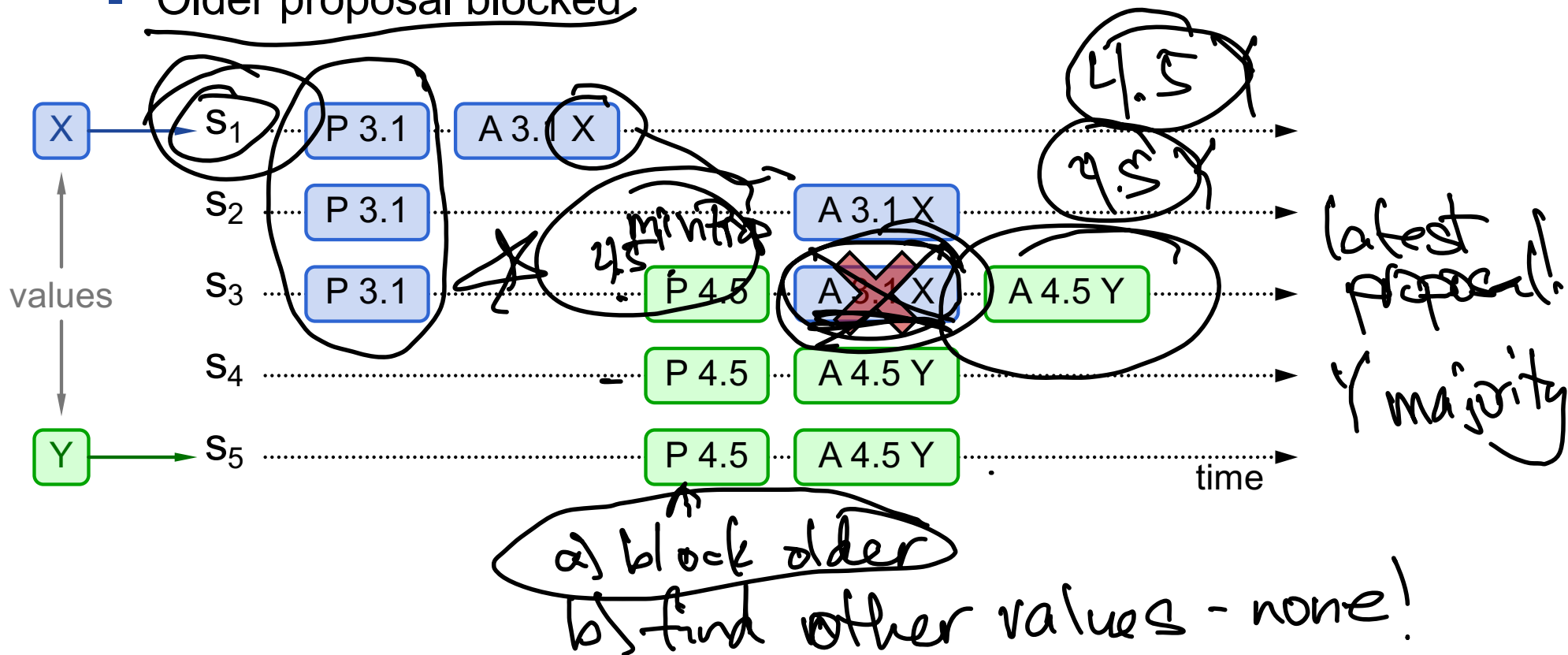


Basic Paxos Example 3

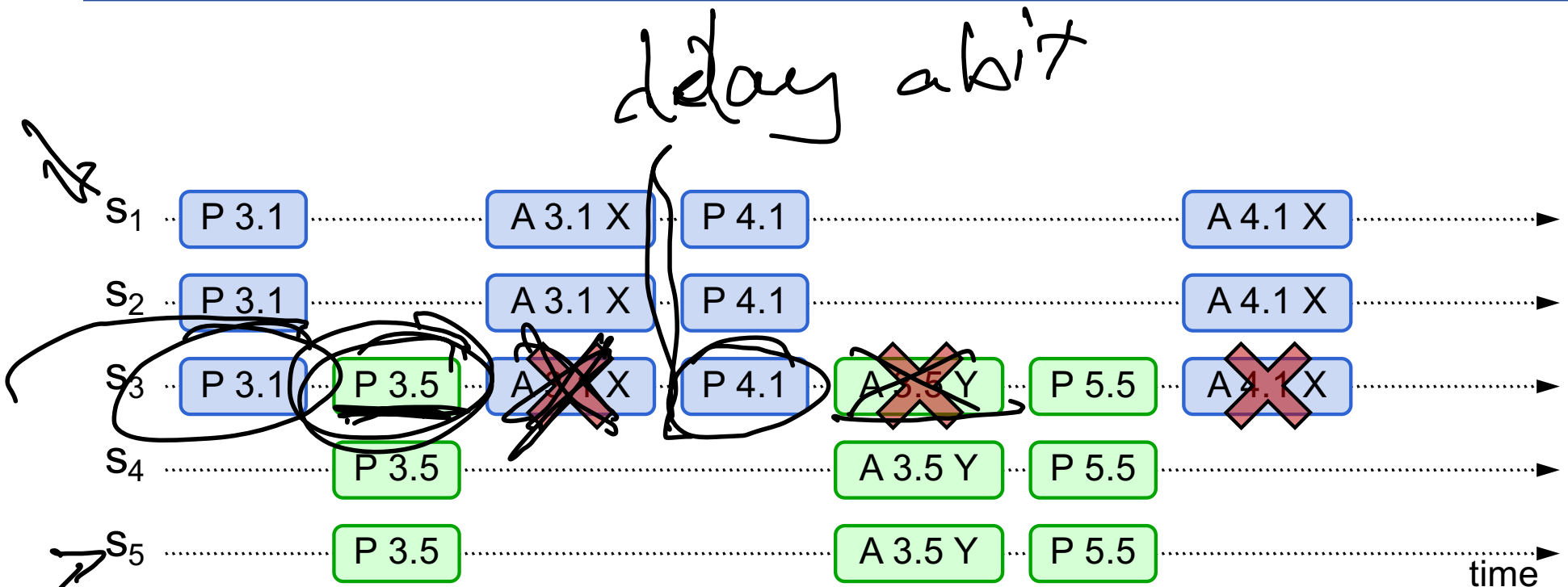
Three possibilities when later proposal prepares:

3. Previous value not chosen, later proposer doesn't see it:

- New proposer chooses its own value
- Older proposal blocked



Liveness: Guaranteed?

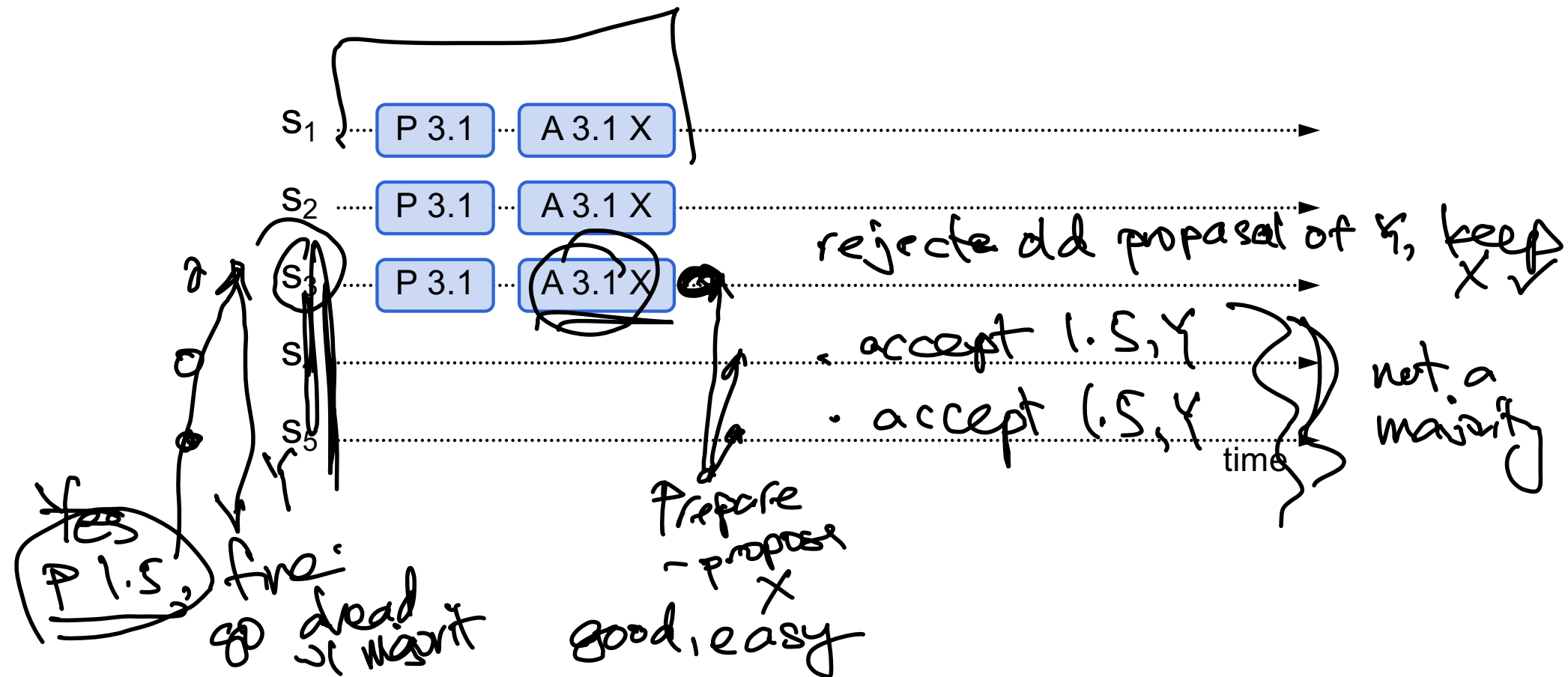


- Keep blocking accept RPC due to later prepares
- One solution: randomized delay before restarting that keep arriving
 - Give other proposers a chance to finish choosing
 - Multi-Paxos will use leader election instead

go to same proposer

Question 1

- After 3.1 X has been accepted, ^{on 3 at 5} can a different server accept a different value of Y? not chosen, just accepted

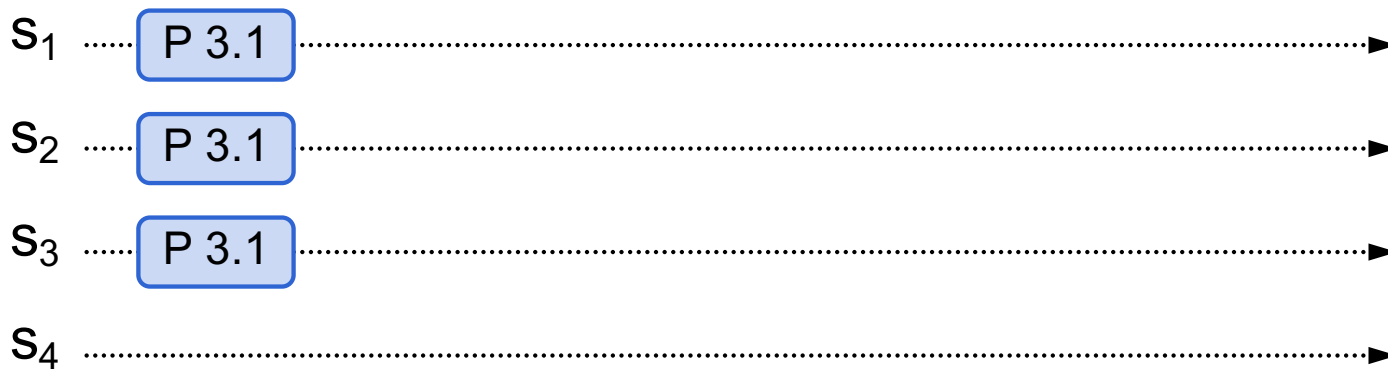


Question 2

- After 3.1 X has been proposed, what if s1 crashes and proposes 3.1 Y; is this safe?

not persistent? 4.1 proper #

needs to be unique



promised not to accept older than ^{time} 3.1
- will accept 3.1 w/ diff. values

Question 3: Safe?

Acceptors

did correct

3) Respond to Prepare(n): ✓

- If $n > \text{minProposal}$ then $\text{minProposal} = n$
- Return(acceptedProposal, acceptedValue)

new Acceptors
broken

3) Respond to Prepare(n): ✓

- If $n > \text{minProposal}$ then $\text{minProposal} = n$
- Return(acceptedProposal, acceptedValue)

6) Respond to Accept(n, value):

- If $n \geq \text{minProposal}$ then
 $\text{acceptedProposal} = \text{minProposal} = n$
 $\text{acceptedValue} = \text{value}$
- Return(minProposal)

6) Respond to Accept(n, value):

- If $n \geq \text{minProposal}$ then
 $\text{acceptedProposal} = n$
 $\text{acceptedValue} = \text{value}$
- Return(minProposal)

~ may have contended diff. acceptors in
prepare vs. accept.
- wouldn't block older proposals

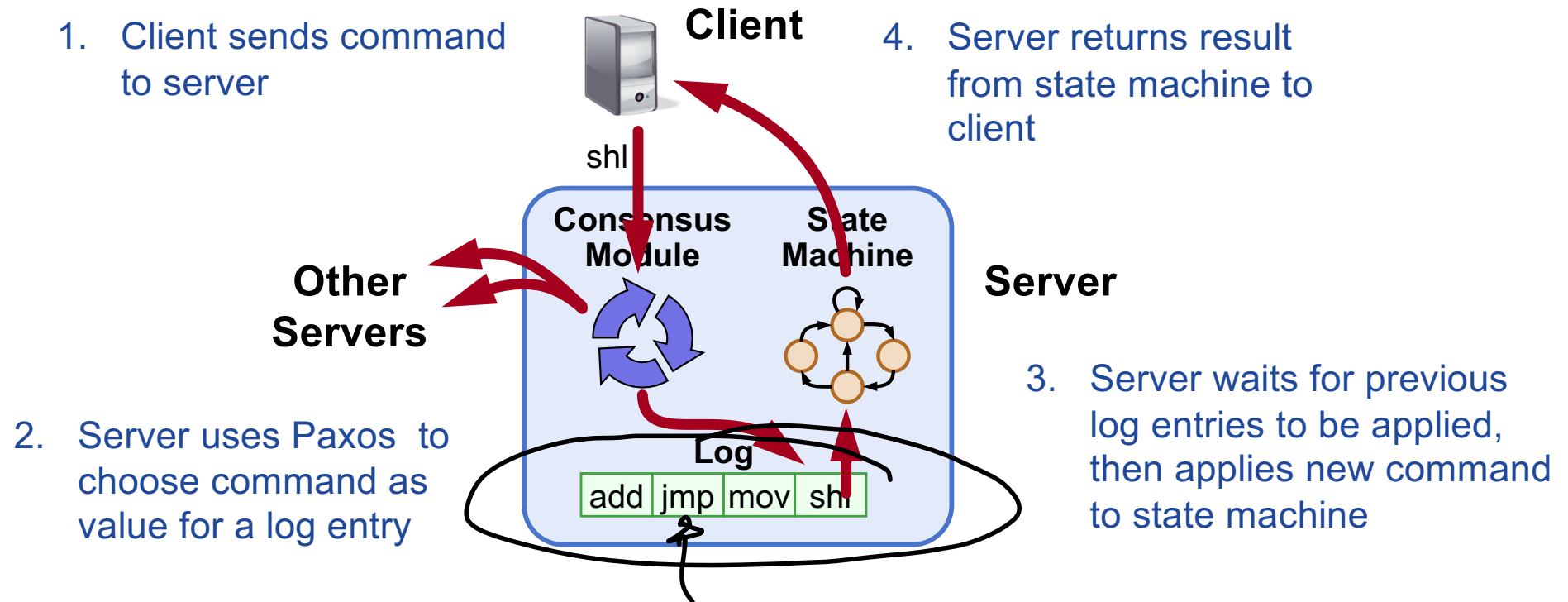
Other Notes

- Only proposer knows which value has been chosen
- If other servers want to know, must execute Paxos with their own proposal

- no-op - value
- informed of correct, pr. - ~~accepted~~

Multi-Paxos

- **Separate instance of Basic Paxos for each entry in the log:**
 - Add **index** argument to Prepare and Accept (selects entry in log)



Multi-Paxos Issues

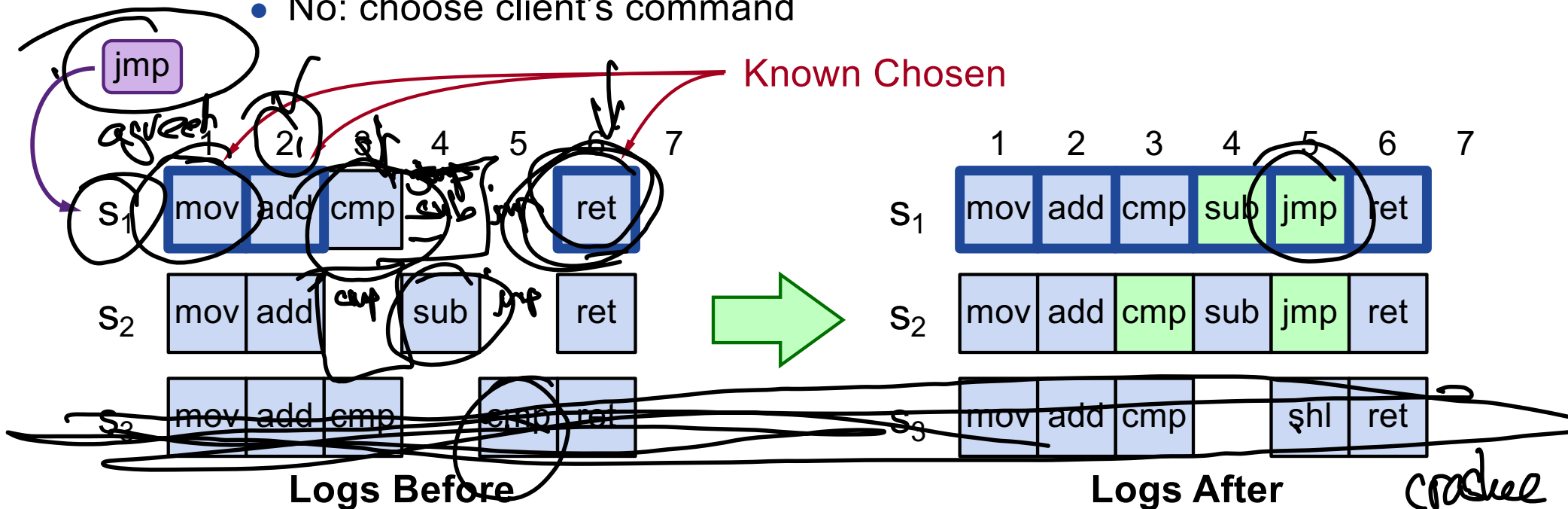
- Which log entry to use for a given client request?
- Performance optimizations:
 - Use leader to reduce proposer conflicts
 - Eliminate most Prepare requests
- Ensuring full replication ~
- Client protocol 、
- Configuration changes

Note: Multi-Paxos not specified precisely in literature

Selecting Log Entries

- When request arrives from client:

- Find first log entry not known to be chosen
- Run Basic Paxos to propose client's command for this index
- Prepare returns acceptedValue?
 - Yes: finish choosing acceptedValue, start again
 - No: choose client's command



Selecting Log Entries

- **Concurrent requests from clients?**
- **Apply requests to state machine concurrently?**

Selecting Log Entries

- **Servers can handle multiple client requests concurrently:**
 - Select different log entries for each
- **Must apply commands to state machine in log order**

Improving Efficiency

- **Using Basic Paxos is inefficient:**

- With multiple concurrent proposers, **conflicts** and restarts are likely (higher load → more conflicts)
- **2 rounds** of RPCs for each value chosen (Prepare, Accept)

Solution:

1. Pick a leader

- At any given time, only one server acts as Proposer

2. Eliminate most Prepare RPCs

- Prepare once for the entire log (not once per entry)
- Most log entries can be chosen in a single round of RPCs

Leader Election

One simple approach from Lamport:

- Let the server with highest ID act as leader
- Each server sends a heartbeat message to every other server every T ms
- If a server hasn't received heartbeat from server with higher ID in last $2T$ ms, it acts as leader:
 - Accepts requests from clients —
 - Acts as proposer and acceptor —
- If server not leader:
 - Rejects client requests (redirect to leader)
 - Acts only as acceptor

- leader?
- know not leader

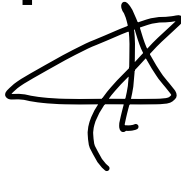
- 2 leaders? problem? no, worse performance
fine correctness

Eliminating Prepares

• Why is Prepare needed? —

- Block old proposals
 - Make proposal numbers refer to the entire log, not just one entry
- Find out about (possibly) chosen values
 - Return highest proposal accepted for current entry
 - Also return **noMoreAccepted**: no proposals accepted for any log entry beyond current one
- If acceptor responds to Prepare with **noMoreAccepted**, skip future Prepares with that acceptor (until Accept rejected)
- Once leader receives **noMoreAccepted** from majority of acceptors, no need for Prepare RPCs
 - Only 1 round of RPCs needed per log entry (Accepts)

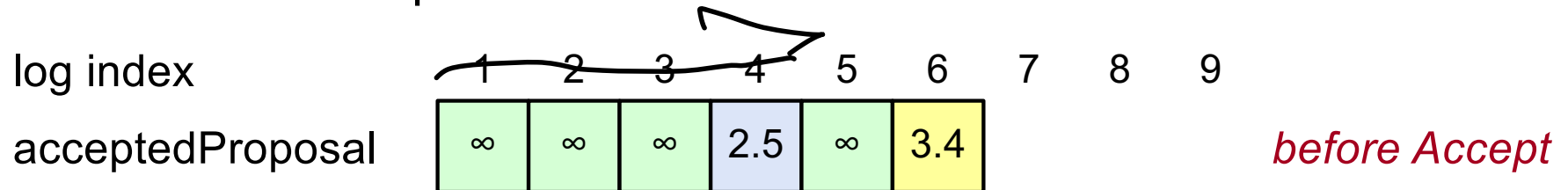
Full Disclosure

- **So far, information flow is incomplete:**
 - Log entries not fully replicated (majority only)
Goal: full replication
 - Only proposer knows when entry is chosen
Goal: all servers know about chosen entries
- **Solution part 1/4: keep retrying Accept RPCs until all acceptors respond (in background)** 
 - Fully replicates most entries
- **Solution part 2/4: track chosen entries**
 - **Mark entries** that are known to be chosen: ✓
 $\text{acceptedProposal}[i] \neq \infty$ 
 - Each server maintains **firstUnchosenIndex**: index of earliest log entry not marked as chosen

Full Disclosure, cont'd

- **Solution part 3/4: proposer tells acceptors about chosen entries**

- Proposer includes its firstUnchosenIndex in Accept RPCs.
- Acceptor marks all entries i chosen if:
 - $i < \text{request.firstUnchosenIndex}$
 - $\text{acceptedProposal}[i] == \text{request.proposal}$
- Result: acceptors know about *most* chosen entries



... Accept(proposal = 3.4, index=8, value = v, firstUnchosenIndex = 7) ...



Still don't have complete information

Full Disclosure, cont'd

- **Solution part 4/4: entries from old leaders**
 - Acceptor returns its firstUnchosenIndex in Accept replies
 - If proposer's firstUnchosenIndex > firstUnchosenIndex from response, then proposer sends **Success** RPC (in background)
- **Success(index, v): notifies acceptor of chosen entry:**
 - $\text{acceptedValue}[\text{index}] = v$
 - $\text{acceptedProposal}[\text{index}] = \infty$
 - return firstUnchosenIndex
 - Proposer sends additional Success RPCs, if needed

Client Protocol

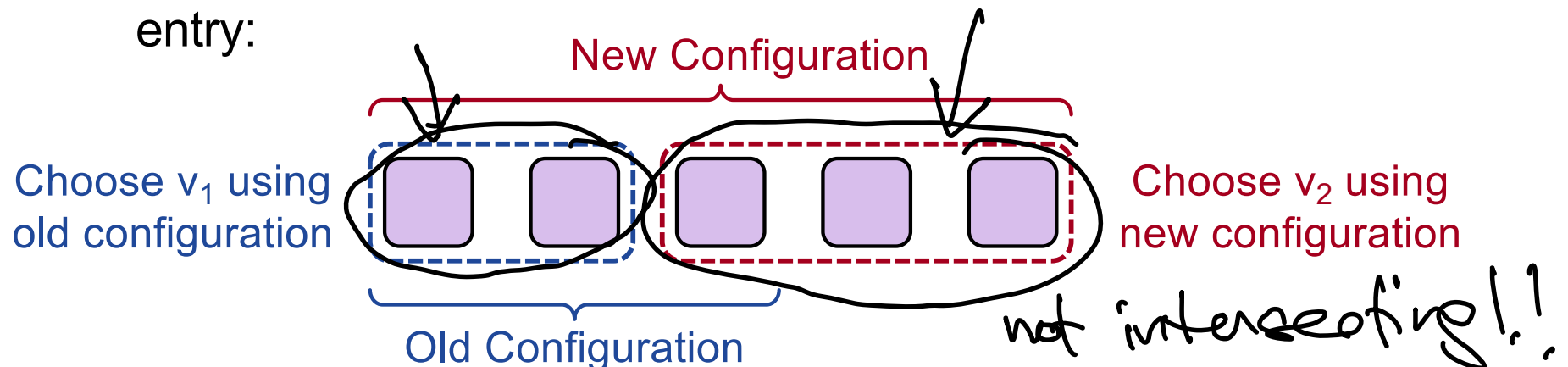
- **Send commands to leader**
 - If leader unknown, contact any server
 - If contacted server not leader, it will redirect to leader
- **Leader does not respond until command has been chosen for log entry and executed by leader's state machine** - variations? - durability needed
- opt: don't order!
- **If request times out (e.g., leader crash):** - opt: not executed
 - Client reissues command to some other server
 - Eventually redirected to new leader
 - Retry request with new leader

Client Protocol, cont'd

- **What if leader crashes after executing command but before responding?**
 - Must not execute command twice
- **Solution: client embeds a unique id in each command**
 - Server includes id in log entry
 - State machine records most recent command executed for each client
 - Before executing command, state machine checks to see if command already executed, if so:
 - Ignore new command
 - Return response from old command
- **Result: **exactly-once semantics** as long as client doesn't crash**

Configuration Changes

- **System configuration:**
 - ID, address for each server
 - Determines what constitutes a majority
- **Consensus mechanism must support changes in the configuration:**
 - Replace failed machine; Change degree of replication
- **Safety requirement:**
 - During configuration changes, it must not be possible for different majorities to choose different values for the same log entry:

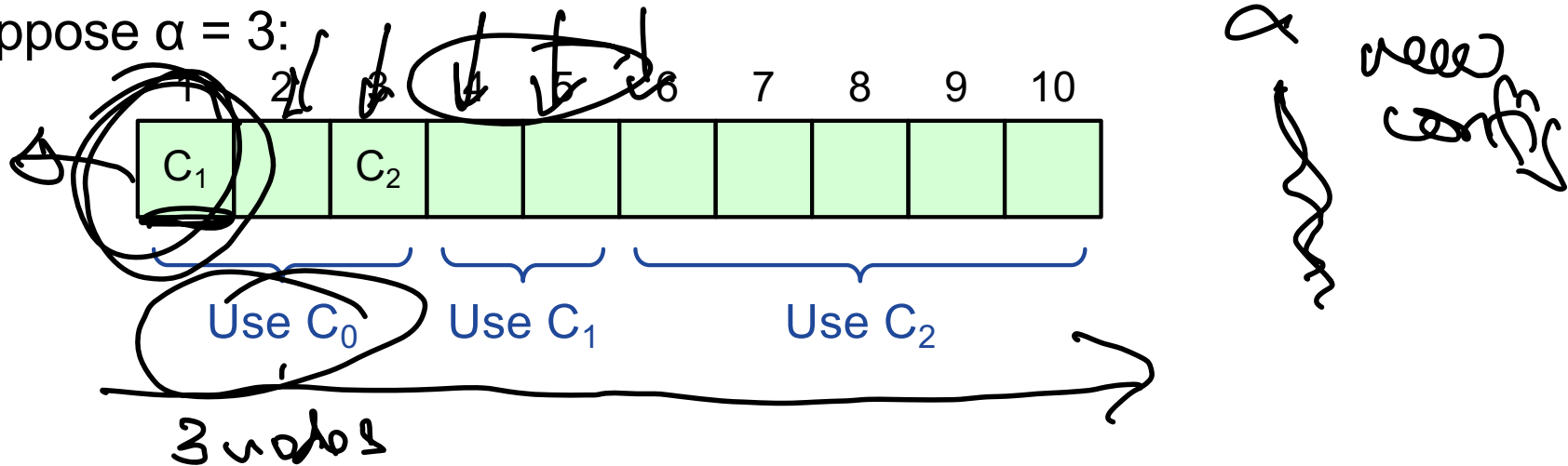


Configuration Changes, cont'd

- **Paxos solution: use the log to manage configuration changes:**

- Configuration is stored as a log entry
- Replicated just like any other log entry
- Configuration for choosing entry i determined by entry $i - \alpha$.

Suppose $\alpha = 3$:



choose α - for system forever
 α : low: limit concurrency - only have α
high: no ops -

Paxos Summary

- **Basic Paxos:**
 - Prepare phase
 - Accept phase
- **Multi-Paxos:**
 - Choosing log entries
 - Leader election
 - Eliminating most Prepare requests
 - Full information propagation
- **Client protocol**
- **Configuration changes**